

Reviewer Pack — Polymatroid Rank Gap in Monotone DNF Depth for k-CLIQUE

Entry 289b9bdc2257 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-07 22:09:59 UTC

Polymatroid Rank Gap in Monotone DNF Depth for k-CLIQUE

Entry ID: 289b9bdc2257 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-07 22:09:59 UTC

1. Conjecture

Field A (mathematical branch): Polymatroid Theory **Field B** (complexity object): Monotone Circuit Depth for k-CLIQUE

Statement:

For any monotone DNF formula computing the k-CLIQUE function on n-vertex graphs, the polymatroid rank function ρ satisfies $\rho \geq n^{\{1/2\}} \cdot k^{\{1/4\}}$, while for general monotone DNF formulas with size $\text{poly}(n)$, $\rho \leq \log n + 2 \log k$.

Rationale (proposer’s reasoning):

Polymatroid rank captures submodular constraints on clause interactions. By linking it to circuit depth, we expose structural bottlenecks in representing clique-like dependencies, which are inherently non-submodular. The gap between k-CLIQUE and general DNF suggests inherent complexity in clique detection.

Taxonomy category: MONOTONE_CLIQUE (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 3e92add9ecb0546d

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

default: $\geq 80\%$ seeds must support, no counterexample

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.00	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.95	UNCERTAIN	SAFE
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
---------	---------	------------	------------------	------------------

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - polymatroid rank k-CLIQUE monotone DNF - polymatroid theory monotone circuits k-CLIQUE complexity bounds - monotone DNF depth polymatroid rank k-CLIQUE

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math

def gcd(a, b):
    while b:
        a, b = b, a % b
    return a

def lcm(a, b):
    return abs(a * b) // gcd(a, b)

def matrix_multiply(A, B):
    m, k, n = len(A), len(B), len(B[0])
    C = [[0] * n for _ in range(m)]
    for i in range(m):
        for j in range(n):
            for l in range(k):
                C[i][j] += A[i][l] * B[l][j]
    return C

def gaussian_elimination(A, b):
    m, n = len(A), len(A[0])
    augmented = [A[i] + [b[i]] for i in range(m)]
    for i in range(n):
        max_row = i
        for j in range(i+1, m):
            if abs(augmented[j][i]) > abs(augmented[max_row][i]):
                max_row = j
        augmented[i], augmented[max_row] = augmented[max_row], augmented[i]
        pivot = augmented[i][i]
        for j in range(n + 1):
            augmented[i][j] /= pivot
        for j in range(m):
            if j != i:
                factor = augmented[j][i]
                for k in range(n + 1):
                    augmented[j][k] -= factor * augmented[i][k]
    return [row[-1] for row in augmented]
```

```

def rank(A):
    m, n = len(A), len(A[0])
    A_copy = [row[:] for row in A]
    return len(gaussian_elimination(A_copy, [0]*n))

def is_clique(G, vertices):
    for u in vertices:
        for v in vertices:
            if u != v and (u, v) not in G and (v, u) not in G:
                return False
    return True

def generate_k_clique_instance(n, k):
    G = {}
    vertices = list(range(n))
    random.shuffle(vertices)
    selected_vertices = vertices[:k]
    for i in range(k):
        for j in range(i+1, k):
            G[(selected_vertices[i], selected_vertices[j])] = True
            G[(selected_vertices[j], selected_vertices[i])] = True
    return G

def generate_random_instance(n):
    G = {}
    vertices = list(range(n))
    random.shuffle(vertices)
    for u in vertices:
        for v in range(u+1, n):
            if random.choice([True, False]):
                G[(u, v)] = True
                G[(v, u)] = True
    return G

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n_values = [5, 10, 15, 20, 30, 40]
    results = []

    for n in n_values:
        k = math.ceil(n ** (1/3))
        G = generate_k_clique_instance(n, k)

        # Construct polymatroid via clause incidence matrix
        A = [[0] * n for _ in range(k)]
        for i in range(k):
            for j in range(n):
                if (i, j) in G:
                    A[i][j] = 1

        rank_k_clique = rank(A)

        # Check  $p \geq n^{1/2} \cdot k^{1/4}$ 
        lower_bound = math.sqrt(n) * k ** (1/4)
        if rank_k_clique < lower_bound:

```

```

        results.append({
            "n": n,
            "k": k,
            "rank": rank_k_clique,
            "lower_bound": lower_bound,
            "conjecture_holds": False,
            "counterexample": f"Rank {rank_k_clique} < {lower_bound}"
        })
    else:
        results.append({
            "n": n,
            "k": k,
            "rank": rank_k_clique,
            "lower_bound": lower_bound,
            "conjecture_holds": True,
            "counterexample": ""
        })

# Check against DNF formulas with size  $\leq n^2$  (using subset lattice polymatroids)
for n in n_values:
    k = math.ceil(n ** (1/3))
    G = generate_random_instance(n)

    # Construct polymatroid via clause incidence matrix
    A = [[0] * n for _ in range(2**n)]
    for i in range(2**n):
        for j in range(n):
            if bin(i)[2:].zfill(n)[j] == '1':
                A[i][j] = 1

    rank_dnf = rank(A)

    # Check  $p \leq \log n + 2 \log k$ 
    upper_bound = math.log(n) + 2 * math.log(k)
    if rank_dnf > upper_bound:
        results.append({
            "n": n,
            "k": k,
            "rank": rank_dnf,
            "upper_bound": upper_bound,
            "conjecture_holds": False,
            "counterexample": f"Rank {rank_dnf} > {upper_bound}"
        })
    else:
        results.append({
            "n": n,
            "k": k,
            "rank": rank_dnf,
            "upper_bound": upper_bound,
            "conjecture_holds": True,
            "counterexample": ""
        })

metric_values = [result["rank"] for result in results]
conjecture_holds = all(result["conjecture_holds"] for result in results)
counterexample = next((result["counterexample"] for result in results if not result["conjecture_holds"]), None)

```

```

    return {
        "metric_name": "polymatroid_rank",
        "metric_value": sum(metric_values) / len(metric_values),
        "instances_tested": len(results),
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    import sys
    seeds = [int(arg) for arg in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3

    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")

    mean_value = sum(result["metric_value"] for result in results) / len(results)
    support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)

    if all(result["conjecture_holds"] for result in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std=0.0 support_fraction=1.0")
    elif any(not result["conjecture_holds"] for result in results):
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"{result['counterexample']}\" first_failing_seed={first_failing_seed}")
    else:
        print("RESULT: INCONCLUSIVE mapping_undefined")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_71a29cd3.py", line 179, in <module>
    result = run_trial(seed)
              ~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_71a29cd3.py", line 104, in run_trial
    rank_k_clique = rank(A)
                    ~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_71a29cd3.py", line 57, in rank
    return len(gaussian_elimination(A_copy, [0]*n))
               ~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_71a29cd3.py", line 46, in gaussian_elimination
    augmented[i][j] /= pivot
ZeroDivisionError: division by zero

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Test crashed with ZeroDivisionError, preventing data collection. No counterexample found, and support fraction cannot be evaluated. | next: Debug gaussian_elimination to avoid division by zero and re-run experiments with validated inputs

11. Audit log (LLM calls)

Total LLM calls: 8

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	107631
2	propose	ollama_remote	qwen3:8b	0	0	20532
3	preregistration	ollama_remote	qwen3:8b	0	0	45220
4	novelty	ollama_remote	qwen3:8b	0	0	34096
5	novelty	ollama_remote	qwen3:8b	0	0	12129
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	16372
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	17791
8	judge	ollama_remote	qwen3:8b	0	0	12954

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 266724 ms total latency. Provider mix: {'ollama_remote': 8}

(full prompt+response transcripts available in `research/audit/289b9bdc2257.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/289b9bdc2257.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/289b9bdc2257.tar.gz` (if generated)